

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Методы трансляции

ОТЧЕТ
к лабораторной работе №2
на тему

ЛЕКСИЧЕСКИЙ АНАЛИЗ

Студент
Преподаватель

Е. С. Кахновский
Н. Ю. Гриценко

Минск 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Теоретические сведения	4
3 Результат выполнения	5
Заключение	8
Список использованных источников	9
Приложение А (обязательное) Листинг кода.....	10

1 ЦЕЛЬ РАБОТЫ

Разработать лексический анализатор подмножества языка программирования, определенного в лабораторной работе 1. Определить лексические правила. Выполнить перевод потока символов в поток лексем (токенов). При определении неверной последовательности символов, необходимо обнаружить эту ошибку и выдать сообщение о ней.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Основная задача лексического анализатора в том, чтобы прочитав входные символы исходного текста программы, сгруппировать их в лексемы, и вывести последовательность токенов для всех лексем исходного текста. Поток токенов подаётся на вход синтаксического анализатора.

Токен – структура, состоящая из имени токена и набора необязательных произвольных атрибутов. Имя токена – абстрактный символ (идентификатор), представляющий тип лексической единицы, например, ключевое слово, название переменной.

Лексема представляет собой последовательность символов исходной программы, которая идентифицируется лексическим анализатором как экземпляр токена.

Шаблон – описание вида, который может принимать лексема токена. Лексический анализатор принимает решение о принадлежности лексемы токenu на основе шаблона [1].

Лексемы могут представлять различные элементы программного кода, такие как идентификаторы, ключевые слова, операторы, константы и разделители. Каждая лексема имеет свое семантическое значение и правила использования в контексте языка программирования.

Идентификаторы служат для уникальной идентификации элементов программы, их объявления и обращения к ним в коде. Они могут состоять из букв, цифр и некоторых специальных символов, в зависимости от языка программирования.

Ключевые слова являются зарезервированными словами, имеющими специальное значение в языке программирования. Они используются для определения особых действий и операций в программе.

Операторы являются символами или комбинациями символов, которые выполняют определенные операции или выражают отношения между значениями.

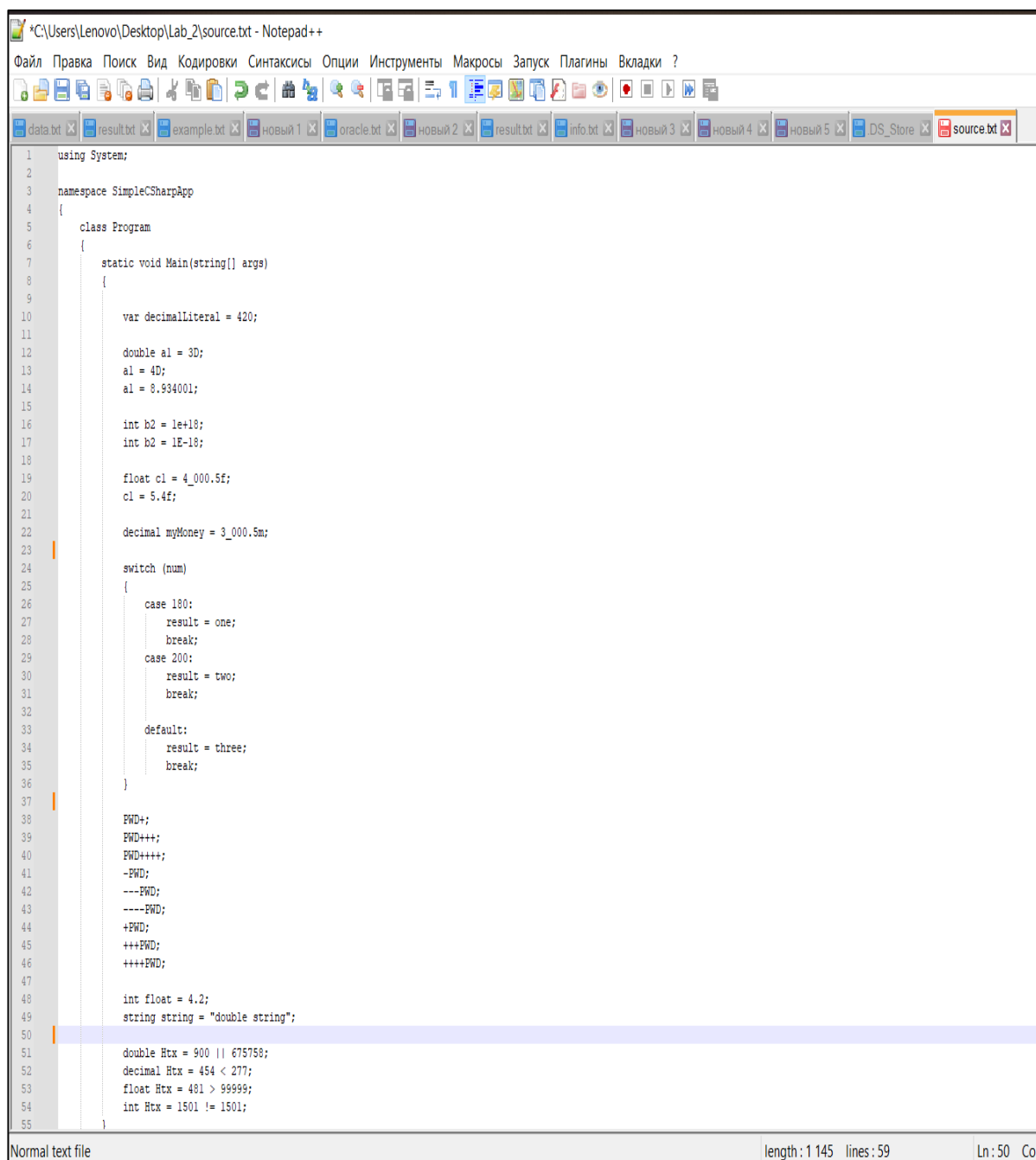
Константы представляют фиксированные значения, которые не могут быть изменены во время выполнения программы.

Разделители используются для разделения элементов программы и обозначения границ различных конструкций.

В контексте языка программирования, лексемы играют важную роль, так как они помогают в создании структуры и синтаксически правильного кода. Компиляторы и интерпретаторы используют лексемы для анализа и выполнения программного кода [2].

3 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

В результате выполнения лабораторной работы был разработан лексический анализатор. Были составлены лексические правила и определение нескольких видов ошибок. В качестве входных данных использовался текстовый файл с языком C# (рисунок 1).



```
1 using System;
2
3 namespace SimpleCSharpApp
4 {
5     class Program
6     {
7         static void Main(string[] args)
8         {
9
10             var decimalLiteral = 420;
11
12             double a1 = 3D;
13             a1 = 4D;
14             a1 = 8.934001;
15
16             int b2 = 1e+18;
17             int b2 = 1E-18;
18
19             float c1 = 4_000.5f;
20             c1 = 5.4f;
21
22             decimal myMoney = 3_000.5m;
23
24             switch (num)
25             {
26                 case 180:
27                     result = one;
28                     break;
29                 case 200:
30                     result = two;
31                     break;
32
33                 default:
34                     result = three;
35                     break;
36             }
37
38             PWD;
39             PWD+++;
40             PWD+++;
41             -PWD;
42             ---PWD;
43             ----PWD;
44             +PWD;
45             +++PWD;
46             ++++PWD;
47
48             int float = 4.2;
49             string string = "double string";
50
51             double Htx = 900 || 675758;
52             decimal Htx = 454 < 277;
53             float Htx = 481 > 99999;
54             int Htx = 1501 != 1501;
55     }
```

Normal text file length: 1145 lines: 59 Ln: 50 Col

Рисунок 1 – Файл с входными данными

В качестве выходных данных были получены списки лексем, разделенные по группам (рисунок 2).

A	B	C	D	E	F	G	H	
ERROR1	ERROR2	ERROR3	ERROR4	ERROR5	ERROR6	KEYWORDS	IDENTIFIERS	OPERATORS
int i=variable =	PWD;	int float =	char str5 = 50853 > 353;	bool var1 = 234 + 324 - int Htx = 12300 && 1201;	using	System	=	
float \$invalid	PWD---	string string =	string value = 8343 < 234 > 123 + 4553;	bool var2 = 576 * 5234 / double Htx = 900 675758;	namespace	SimpleCSharpApp	=	
int 700 =	PWD---		char value = 8343 234 ~ 12353 ^ 4553 & 3222;	bool var3 = 1999 / 2211 decimal Htx = 454 < 277;	class	Program	=	
float ^ =	PWD;		string x4 = 553 + 353;	float Htx = 481 > 99999;	static	Main	=	
int %SomeText =	PWD+++		char SomeChar = 12353 / 4534 + 653 - 453 - 1453 * 234234;	int Htx = 1501 != 1501;	void	args	=	
	PWD+++		string SomeChar = 253-253 * 253;		string	decimalLiteral	=	
	;	-PWO			var	a1	=	
	---PWO				double	a1	=	
	----PWO				int	a1	=	
	;	+PWO			int	b2	=	
	+++PWO				float	b2	=	
	++++PWO				decimal	c1	=	
					char	c1	=	
					char	myMoney	=	
					char	myMoney	=	
					char	ch1	=	
					char	ch2	=	
					char	ch3	=	
					string	ch3	=	
					string	ch4	=	
					bool	str1	=	
					true	str4	<	
					false	check	>	
					object	check	=	
					int	Console	<	
					new	WriteLine	>	
					int	check	=	
					foreach	obj	=	

Рисунок 2 – Списки полученных лексем

Так же была составлена таблица соответствия лексем определенным идентификаторам (рисунок 3).

	A	B
1	Identifier	Value
2	id1	using
3	id2	System
4	id3	namespace
5	id4	SimpleCSharpApp
6	id5	class
7	id6	Program
8	id7	static
9	id8	void
10	id9	Main
11	id10	string
12	id11	args
13	id12	var
14	id13	decimalLiteral
15	id14	420
16	id15	double
17	id16	a1
18	id17	3D
19	id18	4D
20	id19	8.934001
21	id20	int
22	id21	b2
23	id22	1e+18
24	id23	1E-18
25	id24	float
26	id25	c1
27	id26	4_000.5f
28	id27	5.4f
29	id28	decimal

Рисунок 3 – Таблица соответствия идентификаторов лексемам

На основании этой таблицы лексемы были заменены на соответствующие им идентификаторы в исходном файле (рисунок 4).

```
id1 id2;

id3 id4
{
    id5 id6
    {
        id7 id8 id9(id10[] id11)
        {

            id12 id13 = id14;

            id15 id16 = id17;
            id16 = id18;
            id16 = id19;

            id20 id21 = id22;
            id20 id21 = id23;

            id24 id25 = id26;
            id25 = id27;

            id28 id29 = id30;
            id29 = id31;

            id32 id33 = id34;
            id32 id35 = id34;
            id32 id36 = id37;
            id32 id36 = id38;
            id32 id39 = id32(id40);

            id10 id41 = id42;
            id10 id43 = id44;

            id45 id46 = id47;
            id46 = id48;

            id49.id50(id46);

            id51 id52 = id53;
```

Рисунок 4 – Замененные лексемы на идентификаторы

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы был разработан лексический анализатор подмножества языка программирования, определенного в лабораторной работе 1. Определены лексические правила. Выполнен перевод потока символов в поток лексем (токенов). При определении неверной последовательности символов, обнаруживалась ошибка и выдавалось сообщение о ней.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Лексический анализ [Электронный ресурс]. – Режим доступа: <https://wiki.livid.pp.ru/students/sp/lectures/3.html>.

[2] Лексема языка программирования [Электронный ресурс]. – Режим доступа: https://medium.com/@Omg_wolfcii37-8ab35e951762.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг 1 – файл *main.py*.

```
import re
import pandas as pd
from openpyxl import Workbook

# Define regular expressions for a wider range of C# syntax elements

sample_code = ""
try:
    with open('source.txt', 'r') as file:
        sample_code = file.read()
except Exception:
    print("Error reading from file")

token_specs = [
    ('COMMENT', r'//.*'), # Single-line comment
    ('MULTICOMMENT', r'/\*[\\s\\S]*?\\*/'), # Multi-line comment
    ('WHITESPACE', r'\\s+'), # Whitespace
    ('ERROR1',
r'\\b(?:class|int|float|double|string|char|bool|object)\\b\\s*[\\d!@#=%^&*~_\\$][a-zA-Z_]*\\w*\\s*='), # var naming
    ('ERROR2', r'\\b(\\w+\\s*(\\+|\\-){3,})|((\\+|\\-){3,}\\s*\\w+)|;\\s*([+-]
\\s*\\w+)|\\b(\\w+\\s*[+-]);'), #inc, dec
    ('ERROR3',
r'\\b(int|float|double|string|bool|char|object)\\b\\s+\\b(int|float|double|string|bool|char|object)\\b\\s*='), # data types as vars
    ('ERROR4', r'\\b(string|char)\\s+\\w+\\s*=\\s*[\\d]+\\s*([+\\-*/><~&|^]\\s*[\\d]+\\s*)+;'),
    ('ERROR5', r'\\b(bool)\\s+\\w+\\s*=\\s*[\\d]+\\s*([&|^|%+\\-*/]\\s*[\\d]+\\s*)+;'),
    ('ERROR6',
r'\\b(int|float|double|decimal)\\s+\\w+\\s*=\\s*[\\d]+\\s*(((&&)|(\\|\\|\\|)|([><])|(!=))\\s*[\\d]+\\s*)+;'),
    ('STRING', r'["\\'](?:\\.\\.|[\\^"\\\\\\n\\r])*["\\']'), # String literal
    ('NUMBER', r'\\b(?:\\d{1,3}(?:_\\d{3})*(?:\\.\\d+)?|\\d+(?:\\.\\d+)?(?:[eE][-+]?\\d+)?(?:[dfmDFM])?\\b)'), # Integer or decimal number
    ('KEYWORD',
r'\\b(?:class|struct|var|int|float|double|decimal|char|string|public|private|protected|static|foreach|do|if|else|for|while|switch|case|return|void|bool|true|false|null|new|object|using|namespace|default|break|not|in)\\b'), # Keywords
    ('IDENTIFIER', r'\\b[A-Za-z_]\\w*\\b'), # Identifiers
    ('OPERATOR', r'[\\+\\-\\*\\/=%<>^~!&:.|?]+'), # Operators
    ('DELIMITER', r'[{}()\\[\\],;]') # Delimiters
]

# Compile the regular expressions
token_regex = '|'.join('(?P<%s>%s)' % pair for pair in token_specs)
# print(token_regex)
token_re = re.compile(token_regex)

token_dict = {}

# Tokenization function
```

```

def tokenize_enhanced_csharp_code(code):
    tokens = []

    i = 1
    # Tokenize the sample code
    enhanced_tokens = tokenize_enhanced_csharp_code(sample_code)

def categorize_tokens(tokens):
    categories = {
        'ERROR1': [],
        'ERROR2': [],
        'ERROR3': [],
        'ERROR4': [],
        'ERROR5': [],
        'ERROR6': [],
        'KEYWORDS': [],
        'IDENTIFIERS': [],
        'OPERATORS': [],
        'NUMBERS': [],
        'STRINGS': [],
        'DELIMITERS': []
    }

    for token_type, token_value in tokens:
        if token_type == 'ERROR1':
            categories['ERROR1'].append(token_value)
        elif token_type == 'ERROR2':
            categories['ERROR2'].append(token_value)
        elif token_type == 'ERROR3':
            categories['ERROR3'].append(token_value)
        elif token_type == 'ERROR4':
            categories['ERROR4'].append(token_value)
        elif token_type == 'ERROR5':
            categories['ERROR5'].append(token_value)
        elif token_type == 'ERROR6':
            categories['ERROR6'].append(token_value)
        categories['DELIMITERS'].append(token_value)

    # Заполняем пустые списки
    print('\n\n', i, categorized_tokens[i])

for key, value in token_dict.items():
    sample_code = sample_code.replace(value, key)

print(sample_code)

# for key, value in token_dict.items():
#     print(key, value)

for key, value in categorized_tokens.items():
    print(f"{key}: {len(value)}")

max_length = max(len(value) for value in categorized_tokens.values())
filled_tokens = {key: value + [''] * (max_length - len(value)) for key, value
with pd.ExcelWriter('token_data.xlsx') as writer:
    df_token_dict.to_excel(writer, sheet_name='Token Dictionary',
index=False)

print("\n\nDataFrame для таблицы соответствия идентификаторов и их
значений:")
print(df_token_dict)

```